

Execução e Análise de Falha

Ao executar a função `fatorial_quebrado(1)`, o programa falha propositalmente e levanta o seguinte erro:

```
AssertionError: Erro: invariante violado no corpo do loop
```

A asserção que estourou foi a asserção de manutenção do invariante, localizada dentro do corpo do laço `while`:

```
assert f == math.factorial(i) and 0 <= i <= n, "Erro: invariante violado no corpo do loop"
```

Essa asserção verifica se, após cada iteração do laço, o invariante continua válido. O invariante definido no código é:

```
f == i! e 0 <= i <= n
```

Para o caso de teste `n = 1`, inicialmente temos:

```
f = 1  
i = 0
```

Nesse momento, o invariante é verdadeiro, pois:

```
f == 0!  
1 == 1
```

Logo, a asserção de inicialização é satisfeita.

Entretanto, ao entrar no laço, o código executa a seguinte instrução com o bug original:

```
f = f * i
```

Como `i` vale 0 na primeira iteração, o cálculo realizado é:

```
f = 1 * 0  
f = 0
```

Depois disso, o código incrementa o valor de `i`:

```
i = i + 1
```

Assim, `i` passa a valer 1. Nesse ponto, a asserção de manutenção verifica se:

```
f == 1!
```

Mas o valor de `f` é 0, enquanto:

```
1! = 1
```

Portanto, a comparação feita pela asserção é:

```
0 == 1
```

Essa comparação é falsa, então o programa interrompe a execução e levanta o `AssertionError`.

A razão aritmética do erro é que o código multiplica `f` por `i` antes de incrementar `i`. Como o laço começa com `i = 0`, o valor acumulado de `f` é zerado logo na primeira iteração. Dessa forma, o invariante `f == i!` deixa de ser preservado após a atualização das variáveis do laço.

Portanto, a falha ocorre porque, após a primeira iteração, o programa deveria ter `f = 1` para representar `1!`, mas obtém `f = 0`. Isso demonstra que o bug do código original viola a propriedade matemática esperada para o cálculo do fatorial.